WEB PERFORMANCE E QUALIDADE DE ENGENHARIA

Addy Osmani

Performance e Engenharia Medidas

Medir antes de otimizar, restringir antes de construir e provar antes de entregar

OS SETE NÓS DO LOOP addy-osmani

1. **Alinhar Contexto**
2. **Especificar Restrições**
3. **Planejar Tickets**
4. **Construir com TDD**
5. **Runtime e Web Vitals**
6. **Review e Hardening**
7. **Observabilidade e Ship**

SOBRE ESTA OBRA

Este livro ensina a operar o método de Addy Osmani (upstream sob licença MIT) dentro do ecossistema LegoLoop. O método reúne 25 habilidades de engenharia que combatem dois problemas: o achismo de desempenho, em que se otimiza sem medir, e a proliferação de código assistido por IA sem controle de qualidade.

A Parte I apresenta os seis pilares do método. A Parte II descreve o loop mestre de sete estágios e seus seis gates. A Parte III detalha os sub-loops e as habilidades de cada um. A Parte IV desce ao nível de implementação: limiares de Core Web Vitals, os cinco eixos de revisão, os playbooks da skill e o arquivo loops/addy-osmani.loop.json. A Parte V traz receitas e a Parte VI reúne glossário, relatório de conformidade e referência das habilidades.

Fonte primária: docs/manuals/2026-11-02_MANUAL_ADDY_OSMANI_LOOP.md, complementada pelos arquivos upstream da skill aidd-osmani.

Documento gerado deterministicamente a partir das fontes citadas no próprio texto, via pandoc + typst. Toda afirmação técnica desta obra é rastreável a um arquivo nomeado no capítulo correspondente.

Data da compilação: 07/11/2026 · LegoLoop Foundation

Sumário

Como ler este livro **2**
A quem este livro se destina 2
A estrutura da obra 2
As convenções da obra 2
PARTE I — Macro: Medir, Restringir, Provar **3**
1. Dois problemas com a mesma raiz 3
2. Os seis pilares 3
3. O que o leitor leva desta parte 4
PARTE II — Meso: O Loop Mestre de Engenharia **5**
4. Sete estágios, seis gates 5
5. O que os gates protegem 6
6. O loop mestre e o Playbook L 7
PARTE III — Meso: Os Sub-Loops **8**
6. Visão geral 8
7. Alinhamento 8
8. Especificação e Restrições 8
9. Decomposição em Tickets 9
10. Construção Incremental 9
11. Diagnóstico e Recuperação 9
12. Performance Web 10
13. Review e Hardening 10
14. Ciclo de Entrega 10
PARTE IV — Micro: Números, Eixos, Playbooks e Integração **11**
15. Os limiares de Core Web Vitals 11
16. Os cinco eixos de revisão 11
17. Roteamento cognitivo e esforço 12
18. Higiene de contexto e retomada 12
19. Os playbooks da skill 12
20. Integração no LegoLoop 13
PARTE V — Prática: Receitas de Uso **14**
21. Receita: a página de checkout está lenta 14
22. Receita: um bug relatado por cliente 14
23. Receita: auditar uma rota antes de mexer 15
24. Receita: trocar um campo sem derrubar o sistema 15
25. Como escolher a receita 16
PARTE VI — Apêndices **17**
Apêndice A: Glossário 17
Apêndice B: Relatório de conformidade 17
Apêndice C: Referência rápida das habilidades 19

Como ler este livro

A quem este livro se destina

1. **Quem responde pela qualidade de um produto web.** Precisa entender por que "o site está lento" não se resolve com intuição e como orçamentos de performance viram regra. A Parte I responde a isso.
2. **Quem desenvolve com agentes.** Precisa saber qual habilidade chamar, o que cada gate exige e como provar uma correção. As Partes II, III e V são o núcleo para esse perfil.
3. **Quem mantém a plataforma LegoLoop.** Precisa saber onde o pipeline está declarado, quais artefatos cada nó exige e onde o manual e o código divergem. A Parte IV e o Apêndice B são escritos para esse leitor.

A estrutura da obra

Parte	Nível	Conteúdo
I	Macro	Achismo de desempenho, código sem controle e os seis pilares
II	Meso	O loop mestre de sete estágios e seis gates
III	Meso	Os sub-loops e suas habilidades
IV	Micro	Limiares, eixos de revisão, playbooks e integração
V	Prática	Receitas para situações frequentes
VI	Apêndices	Glossário, relatório de conformidade e referência rápida

As convenções da obra

Nomes de habilidades e caminhos de arquivo aparecem em fonte monoespaçada, como `performance-optimization` ou `loops/addy-osmani.loop.json`. Exemplos ilustrativos são marcados como tal no texto. Toda afirmação sobre o LegoLoop é rastreável a um arquivo citado; quando manual e código discordam, o livro segue o código e registra a diferença no Apêndice B.

PARTE I — Macro: Medir, Restringir, Provar

1. Dois problemas com a mesma raiz

O manual apresenta o método de Addy Osmani como a síntese de mais de duas décadas de engenharia web no Google Chrome e no ecossistema JavaScript. O método combate dois problemas que, à primeira vista, parecem distintos.

O primeiro é o **achismo de desempenho**. Alguém diz que a página está lenta, outra pessoa troca uma biblioteca, uma terceira adiciona cache, e ninguém sabe se a página ficou mais rápida, porque ninguém mediu antes. Às vezes ficou mais lenta.

O segundo é a **proliferação de código assistido por IA**. Agentes produzem muito código, rápido. Sem barreiras, esse código cresce em complexidade, ignora limites de segurança e acumula comportamentos que ninguém especificou.

A raiz comum é a ausência de um número ou de uma regra verificável antes de agir. Os dois problemas se resolvem do mesmo jeito: medir, fixar limites por escrito e provar cada mudança.

> **Sem número de partida, otimizar é adivinhar.**

2. Os seis pilares

Pilar 1: medir antes de otimizar

Toda otimização exige telemetria prévia: LCP, INP, CLS, TTFB, perfil de memória, tamanho de bundle. E exige medição posterior com prova de ganho. Uma otimização sem as duas medições não é aceita, mesmo que pareça óbvia.

Pilar 2: desenvolvimento orientado a restrições

Limites de arquitetura, tetos de complexidade ciclomática e contratos de segurança são barreiras ativas, registradas em `CONSTRAINTS.md`. Nenhum agente pode violá-las ou

silenciá-las. A diferença entre uma restrição e uma recomendação é que a restrição bloqueia a entrega.

Pilar 3: ancoragem na documentação oficial

Agentes de IA às vezes inventam sintaxes ou comportamentos de bibliotecas. O método exige que decisões sobre APIs e padrões se baseiem na documentação primária oficial das bibliotecas e frameworks em uso, e não na memória do modelo.

Pilar 4: fatiamento fino e prova por testes

Um bug exige um teste vermelho que reproduza a falha antes da correção. O manual chama esse padrão de *Prove-It*. Funcionalidades avançam em fatias atômicas e testadas, sem stubs e sem quebrar a suíte.

Pilar 5: runtime real e auditoria em cinco eixos

Testes unitários e mocks não substituem a inspeção do navegador de verdade: o DOM, o CSS renderizado, os deslocamentos de layout, os erros de console e o comportamento em viewport real. A revisão final avalia cinco eixos e é acompanhada de contestação adversarial, o *Doubt-Driven Development*.

Pilar 6: entrega observável e segura

O ciclo termina com rastreabilidade: logs estruturados, métricas, traces, um plano de migração do tipo expandir e contrair, e checklists de entrega rigorosos.

> **O PADRÃO PROVE-IT**
>
> Um relato de bug não é corrigido até existir um teste que falha pelo motivo exato do bug. A correção mínima faz esse teste passar, e ele fica na suíte para sempre. Sem o vermelho primeiro, não há como provar que a correção resolveu o problema relatado e não outro.

3. O que o leitor leva desta parte

Pilar	Pergunta que impõe antes de agir
Medir antes de otimizar	Qual é o número hoje?
Restrições	Qual limite esta mudança não pode passar?
Documentação oficial	Onde a documentação diz que funciona assim?
Fatiamento e Prove-It	Qual teste falha agora e passará depois?
Runtime real e cinco eixos	O navegador real confirma? Quem contestou?
Entrega observável	Como saberemos que quebrou em produção?

PARTE II — Meso: O Loop Mestre de Engenharia

4. Sete estágios, seis gates

O loop mestre leva uma ideia à produção em sete estágios. Entre cada par de estágios há um gate binário, de G1 a G6.

1. Alinhamento e Contexto using-agent-skills, interview-me, idea-refine, context-engineering

↓ **G1: alvo e intenção aprovados**

2. Restrições e Especificação constraint-driven, spec-driven, api-and-interface-design

↓ **G2: spec e orçamentos fixados**

3. Fatiamento e Planejamento planning-and-task-breakdown

↓ **G3: DAG de tickets e testes definidos**

4. Construção TDD e UI source-driven, test-driven, frontend-ui, incremental

↓ **G4: suíte TDD 100% verde**

5. Runtime Real e Web Performance browser-testing-with-devtools, performance-optimization

↓ **G5: CWV e runtime auditados**

6. Auditoria, Hardening e Contestação security, doubt-driven, simplification, review

↓ **G6: zero bloqueios em review**

7. Migração, Observabilidade e Ship deprecation, observability, ci-cd, git, shipping

Estágio 1: Alinhamento e Contexto

`using-agent-skills` roteia a intenção para as habilidades certas. `interview-me` e `idea-refine` conduzem a entrevista socrática. `context-engineering` delimita o escopo e registra a baseline: os números e o estado dos testes antes de qualquer mudança.

G1. Alvo e intenção aprovados pelo usuário.

Estágio 2: Restrições e Especificação

`constraint-driven-development` produz o `CONSTRAINTS.md`. `spec-driven-development` fixa os critérios de aceite. `api-and-interface-design` define contratos e tipagem.

G2. Spec e orçamentos de performance fixados. A partir daqui, "rápido" deixa de ser uma opinião e passa a ser um número, por exemplo LCP de até 2,5 segundos.

Estágio 3: Fatiamento e Planejamento

`planning-and-task-breakdown` decompõe a spec em um grafo de tickets verticais.

G3. O DAG de tickets está aprovado e cada ticket tem o teste que vai provar sua conclusão.

Estágio 4: Construção TDD, Diagnóstico e UI

`source-driven-development` busca a documentação primária. `test-driven-development` conduz o ciclo vermelho-verde. `frontend-ui-engineering` e `incremental-implementation` constroem a fatia. Se algo quebra no caminho, `debugging-and-error-recovery` entra como desvio para encontrar a causa raiz.

G4. Suíte TDD 100% verde.

Estágio 5: Runtime Real e Web Performance

`browser-testing-with-devtools` inspeciona o comportamento no navegador real. `performance-optimization` mede os Core Web Vitals e faz o profiling.

G5. CWV e runtime auditados, com números antes e depois.

Estágio 6: Auditoria Multi-Eixo, Hardening e Contestação

`security-and-hardening` cobre o OWASP Top 10. `doubt-driven-development` contesta os achados e as premissas. `code-simplification` e `code-review-and-quality` fecham a revisão.

G6. Zero bloqueios em review.

Estágio 7: Migração, Observabilidade e Ship

`deprecation-and-migration` planeja a migração sem downtime. `observability-and-instrumentation` garante logs e métricas. `ci-cd-and-automation`, `git-workflow-and-versioning` e `shipping-and-launch` levam à entrega.

5. O que os gates protegem

Gate	Sem ele, o risco é
G1	Resolver com perfeição o problema errado
G2	Discutir "está rápido?" sem um número combinado
G3	Tickets que não se sabe quando estão prontos
G4	Medir performance de um código que ainda está quebrado
G5	Auditar segurança de uma versão que ainda vai mudar
G6	Entregar com um achado crítico em aberto

> **A ORDEM ENTRE G4 E G5**
>
> A performance é medida depois da suíte verde, e não antes, porque medir um código com testes falhando produz números de um comportamento que ainda vai mudar. A mesma lógica coloca a auditoria de segurança depois da medição: as otimizações do estágio 5 podem introduzir problemas que o estágio 6 precisa ver.

6. O loop mestre e o Playbook L

A skill `aidd-osmani` tem sua própria versão do fluxo completo, o Playbook L, acionado por pedidos como "fluxo completo na rota X". Ele parte de uma rota existente, e não de uma ideia, e por isso começa auditando.

Playbook L, estágio	Parada	Equivale no loop mestre
1. Alvo	P1	Estágio 1, alinhamento e baseline
2. Auditoria	P2	Estágio 6, antecipado e só leitura
3. Spec e plano	P3	Estágios 2 e 3
4. Execução	P4	Estágio 4, um ticket por vez
5. Prova e entrega	—	Estágios 5 a 7

Cada estágio do Playbook L respeita o limite de três a cinco passos e grava o `ESTADO.md` ao terminar. O fluxo acaba com commits locais e um texto de PR proposto; push e PR só acontecem com ordem do usuário.

PARTE III — Meso: Os Sub-Loops

6. Visão geral

O manual decompõe o ecossistema de Addy Osmani em oito sub-loops, cada um com entradas e saídas contratadas. O arquivo do loop no repositório declara sete deles como nós; o oitavo, `osmani-debugging-recovery`, funciona como desvio do estágio de construção. O Apêndice B registra essa diferença.

Sub-loop	Entrada	Saída contratada
`osmani-alignment`	Ideia ou demanda bruta	Intenção com alvos e baseline
`osmani-spec-constraints`	Intenção validada	Spec e `CONSTRAINTS.md` com orçamentos
`osmani-task-breakdown`	Spec e contratos	Plano com DAG e testes por ticket
`osmani-incremental-build`	Fatias do plano	Código sem stubs e testes verdes
`osmani-debugging-recovery`	Falha ou regressão	Teste vermelho, patch e suíte verde
`osmani-performance-webperf`	Rota em runtime local	Relatório com métricas antes e depois
`osmani-review-hardening`	Código e diff consolidado	Auditoria em 5 eixos e diff simplificado
`osmani-ship-lifecycle`	Código auditado	ADRs, instrumentação e release

7. Alinhamento

Habilidades: `using-agent-skills`, `interview-me`, `idea-refine` e `context-engineering`.

O sub-loop desambigua a intenção, conduz a ideação socrática e isola o contexto necessário. A saída é um documento de intenção em `docs/intents/` com os alvos resolvidos e a baseline inicial. No manual, este sub-loop corresponde ao Playbook I, Definir.

8. Especificação e Restrições

Habilidades: `constraint-driven-development`, `spec-driven-development` e `api-and-interface-design`.

O sub-loop fixa orçamentos de performance, barreiras de qualidade e contratos de interface. As saídas são a spec em `docs/specs/` e o `CONSTRAINTS.md`. As receitas associadas são `/spec` e `/constraints`.

> **SPEC E RESTRIÇÃO NÃO SÃO A MESMA COISA**
>
> A spec diz o que o sistema faz: "o usuário vê o resumo do pedido antes de pagar". A restrição diz o que nenhuma mudança pode violar: "LCP da rota de checkout até 2,5 segundos" ou "nenhuma função acima de complexidade ciclomática 10". A spec muda a cada funcionalidade; a restrição atravessa todas.

9. Decomposição em Tickets

Habilidade: `planning-and-task-breakdown`. **Receita:** `/plan`.

A spec vira fatias verticais finas, os *tracer bullets*, ordenadas em um DAG. Segundo o Playbook K da skill, cada ticket tem identificador, título, critério binário, bloqueios, arquivos envolvidos e o teste que prova sua conclusão.

10. Construção Incremental

Habilidades: `source-driven-development`, `test-driven-development`, `incremental-implementation` e `frontend-ui-engineering`. **Papel:** engenheiro de testes. **Receitas:** `/build` e `/test`.

A construção segue a documentação oficial, o ciclo vermelho-verde e implementa UI acessível. Os checklists de apoio são `testing-patterns.md` e `accessibility-checklist.md`, este baseado na WCAG 2.1. A saída é código de produção real, sem stubs, com testes unitários e de componente verdes.

11. Diagnóstico e Recuperação

Habilidades: `debugging-and-error-recovery` e `test-driven-development` no padrão Prove-It. **Papel:** engenheiro de testes.

Este sub-loop corresponde ao Playbook B, Corrigir. A skill descreve a sequência: reproduzir e registrar requisição, resposta e status; sem reprodução, parar. Depois, um teste que falha pelo motivo certo (gate vermelho), a correção mínima da causa raiz (gate verde) e a confirmação de que o sintoma original não se reproduz mais.

12. Performance Web

Habilidades: `browser-testing-with-devtools` e `performance-optimization`. **Papel:** auditor de performance web. **Receita:** `/webperf`.

O sub-loop mede Core Web Vitals, faz profiling de memória e valida no DevTools. A saída é um relatório em `docs/reports/` com métricas objetivas antes e depois. A Parte IV traz os limiares numéricos que o checklist de performance usa.

13. Review e Hardening

Habilidades: `code-review-and-quality`, `security-and-hardening`, `doubt-driven-development` e `code-simplification`. **Papéis:** revisor de código e auditor de segurança. **Receitas:** `/review` e `/code-simplify`.

O sub-loop audita em cinco eixos, cobre o OWASP Top 10, contesta os achados e simplifica sem perda de comportamento. Corresponde aos Playbooks A (Auditar), D (Simplificar) e F (Proteger). No Playbook F, cada correção de segurança precisa de um teste negativo que prove o bloqueio.

14. Ciclo de Entrega

Habilidades: `deprecation-and-migration`, `observability-and-instrumentation`, `documentation-and-adrs`, `git-workflow-and-versioning`, `ci-cd-and-automation` e `shipping-and-launch`. **Receita:** `/ship`.

O sub-loop planeja a migração sem downtime, ativa a observabilidade, automatiza o CI/CD e faz o release. Os checklists são `observability-checklist.md` e `definition-of-done.md`. As saídas são ADRs em `docs/adrs/`, instrumentação validada, commits semânticos e checklist de release completo. Corresponde aos Playbooks G (Entregar), H (Migrar) e L (Ciclo completo).

PARTE IV — Micro: Números, Eixos, Playbooks e Integração

15. Os limiares de Core Web Vitals

O arquivo `references/performance-checklist.md`, no upstream da skill `aidd-osmani`, fixa as faixas usadas para classificar as três métricas de experiência do usuário.

Métrica	Bom	Precisa melhorar	Ruim
LCP, maior elemento visível	≤ 2,5 s	≤ 4,0 s	> 4,0 s
INP, resposta à interação	≤ 200 ms	≤ 500 ms	> 500 ms
CLS, deslocamento acumulado	≤ 0,1	≤ 0,25	> 0,25

O mesmo checklist liga cada métrica a ações concretas. Imagens e elementos `<source>` com largura e altura explícitas evitam CLS. A imagem principal da página usa `fetchpriority="high"` e não é carregada sob demanda, para não atrasar o LCP. Tarefas longas, acima de 50 ms, são quebradas para liberar a thread principal, que o checklist aponta como a principal alavanca do INP.

16. Os cinco eixos de revisão

O upstream da habilidade `code-review-and-quality` define cinco eixos e afirma que toda mudança é revisada antes do merge, sem exceções.

Eixo	Pergunta central
Correção	O código faz o que a spec pede, inclusive nas bordas?
Legibilidade e simplicidade	Outra pessoa entende sem explicação?
Arquitetura	A mudança respeita os limites dos módulos?
Segurança	Introduz alguma vulnerabilidade?
Performance	Introduz algum problema de desempenho?

Os eixos de segurança e performance remetem às habilidades especializadas, `security-and-hardening` e `performance-optimization`, para a análise detalhada.

17. Roteamento cognitivo e esforço

Sub-loop	Papel cognitivo	Modelo	Esforço
Alinhamento	Raciocínio socrático	Sonnet ou Code-Pro	Médio
Spec e restrições	Síntese de contratos	Sonnet	Alto
Tickets	Planejamento de DAG	Code-Pro	Médio
Construção	Engenharia TDD	Sonnet	Alto
Diagnóstico	Isolamento de falhas	Sonnet	Alto
Performance	Telemetria e profiling	Code-Pro	Médio
Review	Auditoria adversarial	Sonnet	Alto
Entrega	Governança e documentação	Code-Pro	Baixo a médio

18. Higiene de contexto e retomada

1. **Janela de alinhamento.** Intenção, restrições, spec e plano acontecem em uma janela de contexto progressiva.
2. **Contexto estéril por ticket.** Cada ticket é implementado apenas com a spec e as fronteiras do ticket, sem rascunhos anteriores.
3. **Arquivo de estado.** Tarefas longas mantêm `docs/osmani/<alvo>/ESTADO.md` com o estágio atual, os gates satisfeitos com evidência (`arquivo:linha` ou log) e os tickets pendentes. Se o arquivo já existe, a skill o lê, informa o estágio e continua dali.

19. Os playbooks da skill

A skill `aidd-osmani` organiza as habilidades em doze playbooks. Em cada um, o agente para em cada gate e resume em uma linha a passagem entre fases.

Playbook	Nome	Uso típico
A	Auditar	Varredura somente leitura por eixo, com contestação
B	Corrigir	Bug com reprodução, Prove-It e correção mínima
C	Construir	Funcionalidade da entrevista à revisão
D	Simplificar	Menos código com comportamento idêntico
E	Otimizar	Medir, atacar o maior gargalo, medir de novo
F	Proteger	Segurança com teste negativo por correção
G	Entregar	Review final, observabilidade, ADR e ship
H	Migrar	Expandir e contrair com testes e rollback
I	Definir	Entrevista até spec ou decisão de não fazer
J	Documentar	ADRs e contratos
K	Planejar	Tickets verticais com critério binário
L	Ciclo completo	Encadeia os demais em cinco estágios, P1 a P4

O Playbook L é comparado ao loop mestre no capítulo 6.

20. Integração no LegoLoop

O arquivo `loops/addy-osmani.loop.json` segue o esquema `dag.v1`, tem gatilho manual e declara sete nós em cadeia linear, todos apontando para `skills/aidd-osmani`.

Nó	Artefato exigido pelo gate	Modelo sugerido
`subloop-1-alinhar-contexto`	`docs/intents/INTENT.md`	sonnet, ou haiku
`subloop-2-especificar-restricoes`	`docs/specs/SPEC.md`	não declarado
`subloop-3-planejar-tickets`	`docs/plans/PLAN.md`	não declarado
`subloop-4-construir-tdd`	`docs/reports/TDD_REPORT.md`	sonnet, ou qwen
`subloop-5-runtime-webperf`	`docs/reports/PERF_REPORT.md`	não declarado
`subloop-6-review-hardening`	`docs/reports/CODE_REVIEW.md`	sonnet, ou haiku
`subloop-7-observabilidade-ship`	`docs/reports/SHIP_REPORT.md`	não declarado

Cada gate é do tipo `file_exists(...)`: confirma que o artefato existe, não que está correto. No Studio, o template se chama "Addy Osmani Web Performance & Rigor Loop", em `internal/studio/templates.go`.

Os comandos reais do CLI para operar a esteira são:

```
legoloop validate loops/addy-osmani.loop.json
legoloop run loops/addy-osmani.loop.json --var target_route=/checkout
legoloop studio        # padrão: 127.0.0.1:7878
```

Em 07/11/2026, o `validate` respondeu `7 nós válidos, sem ciclos`, em sete camadas de um nó cada.

PARTE V — Prática: Receitas de Uso

As receitas combinam sub-loops e playbooks para situações frequentes. Rotas, números e mensagens são ilustrativos.

21. Receita: a página de checkout está lenta

Quando usar. Usuários reclamam da demora no checkout. Ninguém mediu nada ainda.

1. Rode `performance-optimization` só para medir. Registre LCP, INP e CLS da rota `/checkout` em condições reproduzíveis.
2. Compare com os limiares do checklist. Suponha um LCP de 4,6 s: está na faixa ruim.
3. Fixe o orçamento em `CONSTRAINTS.md`, por exemplo LCP de até 2,5 s.
4. Ataque o maior gargalo, uma mudança por vez, com `incremental-implementation`.
5. Meça de novo depois de cada mudança.
6. Rode `browser-testing-with-devtools` para confirmar no navegador real que nada quebrou visualmente.

Mudança (ilustrativa)	LCP antes	LCP depois
Imagem principal com `fetchpriority=high`	4,6 s	3,4 s
Remoção de biblioteca de carrossel	3,4 s	2,7 s
Fonte com pré-carregamento	2,7 s	2,3 s

Erro comum. Aplicar as três mudanças juntas e medir só no fim. O ganho total aparece, mas não se sabe qual mudança contribuiu, nem se alguma piorou o resultado.

22. Receita: um bug relatado por cliente

Quando usar. "O cupom de desconto é aplicado duas vezes."

1. Rode `debugging-and-error-recovery`. Reproduza e registre requisição, resposta e status. Sem reprodução, pare e peça dados.
2. Escreva um teste que falha pelo motivo exato: aplicar o cupom duas vezes resulta em desconto dobrado. Este é o gate vermelho.

3. Corrija a causa raiz com a menor mudança possível. O teste fica verde, junto com o restante da suíte da rota.
4. Se a correção toca autenticação, dados ou permissões, rode `security-and-hardening`.
5. Rode `code-review-and-quality` e `doubt-driven-development` sobre o diff.
6. Confirme que o sintoma original não se reproduz mais.

> **TESTE QUE FALHA PELO MOTIVO CERTO**
>
> Um teste que falha por erro de compilação ou por fixture quebrada não é o vermelho do Prove-It. O vermelho válido mostra o sintoma do bug: o valor esperado do desconto e o valor dobrado lado a lado na mensagem de falha.

23. Receita: auditar uma rota antes de mexer

Quando usar. A equipe vai assumir uma rota que não conhece e quer saber o que está errado antes de mudar qualquer coisa.

1. Playbook A, passo 1: resolva a rota em artefatos e rode os testes existentes para registrar a baseline.
2. Defina a barra de qualidade com `constraint-driven-development`, em modo leitura.
3. Varra um eixo de cada vez: correção e arquitetura, contrato, segurança, testes, UI em tela real, performance, visibilidade e complexidade.
4. Rode `doubt-driven-development` sobre os achados críticos e altos.
5. Converta os achados confirmados em tickets verticais com `planning-and-task-breakdown`.

Nenhum passo desta receita altera código. Achados de complexidade são apontados, não aplicados.

24. Receita: trocar um campo sem derrubar o sistema

Quando usar. A coluna `nome_completo` precisa virar `nome` e `sobrenome`, e há clientes em produção.

1. Rode `deprecation-and-migration` e planeje em expandir e contrair.
2. **Expandir:** crie os campos novos e passe a gravar nos dois formatos.

3. Migre os dados antigos em lotes, com testes de equivalência.
4. Passe a ler dos campos novos, mantendo os antigos.
5. **Contrair:** depois de um período de observação sem erros, remova o campo antigo.
6. Registre a decisão em uma ADR com `documentation-and-adrs`.

Cada passo é implantável e reversível isoladamente. É isso que permite a migração sem tempo de indisponibilidade.

25. Como escolher a receita

Situação	Receita	Playbook da skill
Lentidão percebida, sem número	21	E, Otimizar
Bug relatado com sintoma claro	22	B, Corrigir
Rota desconhecida, antes de mexer	23	A, Auditar
Mudança de esquema com clientes ativos	24	H, Migrar
Rota inteira, da auditoria à entrega	—	L, Ciclo completo

PARTE VI — Apêndices

Apêndice A: Glossário

Baseline. Medição do estado atual, feita antes de qualquer mudança, que serve de comparação para provar ganho ou regressão.

CLS. Cumulative Layout Shift; mede quanto o conteúdo se desloca enquanto a página carrega.

CONSTRAINTS.md. Arquivo que registra as restrições ativas do projeto: orçamentos de performance, tetos de complexidade e contratos de segurança.

Doubt-Driven Development. Contestação adversarial de achados e premissas antes de aceitá-los.

Expandir e contrair. Estratégia de migração em que o novo formato convive com o antigo até a remoção segura deste.

INP. Interaction to Next Paint; mede o tempo entre uma interação e a próxima atualização visual.

LCP. Largest Contentful Paint; mede quando o maior elemento visível termina de ser desenhado.

Orçamento de performance. Limite numérico que nenhuma mudança pode ultrapassar.

Prove-It. Padrão em que a correção de um bug só começa depois de um teste que falha pelo motivo do bug.

Tracer bullet. Ticket vertical que entrega um comportamento observável atravessando todas as camadas.

Apêndice B: Relatório de conformidade

Comparação entre o manual de 02/11/2026 e o repositório em 07/11/2026.

Ponto verificado	Manual	Código ou upstream	Situação
Número de sub-loops	8	7 nós no JSON	Diverge
Nome do loop no JSON	Não citado	Diz "(8 Sub-Loops)" com 7 nós	Diverge
Nó de diagnóstico	`osmani-debugging-recovery`	Ausente no JSON	Diverge
Validação do DAG	Não citada	7 nós, sem ciclos	Conforme
Nome do template no Studio	"...& Engineering Loop"	"...& Rigor Loop"	Diverge
Parâmetros do `run`	`--param`, `--verbose`	Não existem; usa-se `--var`	Diverge
Endereço do Studio	`localhost:3000`	`127.0.0.1:7878`	Diverge
Quinto eixo de revisão	Manutenibilidade	Legibilidade e simplicidade	Diverge
`metrics-guide.md`	Citado	Não encontrado no upstream	Ausente
`profiling-guide.md`	Citado	Não encontrado no upstream	Ausente
`threat-modeling.md`	Citado	Seção do `security-checklist.md`	Diverge
`idea-refiner.py`	Citado	Não encontrado no upstream	Ausente
Total de habilidades	25	25 em `upstream/skills/`	Conforme

Leitura do relatório. As divergências de operação são as mais importantes para o leitor: os comandos do manual com `--param` e `--verbose` falham no CLI atual, que aceita `--var chave=valor`; o Studio sobe por padrão em `127.0.0.1:7878`. Este livro usa a forma real em todos os exemplos. O nó de diagnóstico ausente no JSON é coerente com o papel de desvio que o diagnóstico tem no loop mestre, mas o nome do pipeline ainda anuncia oito sub-loops.

Apêndice C: Referência rápida das habilidades

Habilidade	Estágio	Função resumida
`using-agent-skills`	1	Roteia a intenção para as habilidades
`interview-me`	1	Entrevista até entender a intenção real
`idea-refine`	1	Refina um conceito cru em opções
`context-engineering`	1	Escopo cirúrgico e baseline
`constraint-driven-development`	2	Restrições ativas em `CONSTRAINTS.md`
`spec-driven-development`	2	Spec com critérios de aceite
`api-and-interface-design`	2	Contratos de API e tipagem
`planning-and-task-breakdown`	3	DAG de tickets verticais
`source-driven-development`	4	Decisões ancoradas em docs oficiais
`test-driven-development`	4	Vermelho e verde; Prove-It
`incremental-implementation`	4	Fatias pequenas e testadas
`frontend-ui-engineering`	4	UI acessível e responsiva
`debugging-and-error-recovery`	4	Causa raiz com reprodução
`browser-testing-with-devtools`	5	Inspeção no navegador real
`performance-optimization`	5	Medição, profiling e otimização
`security-and-hardening`	6	OWASP Top 10 e endurecimento
`doubt-driven-development`	6	Contestação adversarial
`code-simplification`	6	Menos código, mesmo comportamento
`code-review-and-quality`	6	Revisão em cinco eixos
`deprecation-and-migration`	7	Expandir e contrair
`observability-and-instrumentation`	7	Logs, métricas e traces
`documentation-and-adrs`	7	ADRs e documentação
`git-workflow-and-versioning`	7	Commits atômicos e versionamento
`ci-cd-and-automation`	7	Pipelines e gates automáticos
`shipping-and-launch`	7	Checklist, rollout e rollback